Réalisation d'un correcteur linguistique

Nathan T.

13. June 2025

Table des matières

1 Introduction **2**
1.1 Historique 2
1.2 Définitions 2
1.3 Objectifs et critères 2
2 Obtention et préparation des données **3**
2.1 Origine du corpus 3
2.2 Préparation du corpus 3
3 Approche algorithmique **5**
4 Utilisation d'un réseau de neurones **7**
4.1 Tokenisation & Embedding 7
5 Conclusion **8**
6 Annexe **9**
Bibliographie **10**

1 Introduction

Construire un correcteur syntaxique et grammatical flexible. On utilisera deux approches distinctes : une algorithmique et une basée sur un réseau de neurones.

1.1 Historique

Pour un bref historique, le chapitre introduisant les motivations dans la thèse [1] synthétise l'évolution des premiers algorithmes et outils reliés au traitement des langues naturelles.

1.2 Définitions

Tout d'abord, il est nécessaire de définir ce que l'on souhaite réaliser, et cela passe partiellement par la définition précise de ce qu'on appelle syntaxe, grammaire et orthographe.

Nous utiliserons systématiquement comme source le CNRTL [2], ou *Centre national de ressources textuelles et lexicales* comme dictionnaire.

La syntaxe

Selon le CNRTL[1] , la syntaxe est la *partie de la grammaire traditionnelle qui étudie les relations entre les mots constituant une proposition ou une phrase, leurs combinaisons, et les règles qui président à ces relations, à ces combinaisons.*

1.3 Objectifs et critères

Afin de mieux cerner les objectifs de ce TIPE et d'évaluer les différents modèles et solutions proposés, nous utiliserons différents critères, listés ci-dessous par ordre d'importance.

La **pertinence des corrections**. Il est hautement souhaitable de limiter autant que possible les corrections ou bien incorrectes ou redondantes. Il s'agit donc dans un premier temps de détecter le moins de de faux positifs possibles.

La **détection d'un maximum d'erreur**. Repérer hypothétiquement toutes les erreurs, et traiter le plus de cas différents pour avoir une couverture le plus général possible de la langue. Cet objectif ne doit en revanche en aucun cas contrevenir au premier critère et réduire la pertinence des corrections.

La **suggestion de reformulations**. Un phrase peut-être grammaticalement et syntaxiquement correcte quand bien-même elle est alambiquée, interminable voire incompréhensible. Ce critère s'appliquera exclusivement au modèle neuronal proposé.

[1] https://www.cnrtl.fr/definition/syntaxe

2 Obtention et préparation des données

Il est dans un premier temps nécessaire d'avoir accéder à un corpus suffisament important de mots. Celui le plus complet que j'ai trouvé est le Wiktionnaire[2] dont les données sont librement accessibles[3] en intégralité.

Pour ma part, j'ai utilisé le travail réalisé par kaikki.org[4], avec le contenu du Wiktionnaire datant du 20 avril 2025.

2.1 Origine du corpus

Corpus	Nombre de mots	Date	Remarques
Lefff	451962	2006	Complet, de bonne qualité. Forme part d'Alexina, et une variante, le Leffe, existe pour l'espagnol.
Lexique 383	125396	Création en 2019 - mis à jour jusqu'à maintenant	Relativement complet, d'excellente qualité
Wiktionnaire	1859114	Dump du 20 avril 2025	Extrêmement complet, de très bonne qualité

2.2 Préparation du corpus

Dans un premier temps, on doit avoir un moyen de vérifier qu'un mot est correctement orthographié de manière efficace. On remarque que dans le wiktionnaire, de nombreux symboles sont présents jusqu'à certains émojis.

Un premier tri implique donc de ne garder que certains mots. Pour cela, on peut ne garder que les caractères alphanumériques au sens large.

```py
def est_valide(c):
  return c.isalpha() or c == '-'
```

Toutefois, il reste encore, malgré cette première étape, un nombre assez conséquent de caractères utilisés, et le chargement de la trie prend environ 5 secondes.

acueilrnyopédmgvsèhftbAjkCD-ROMqwExTINBçàLïzHPUSYùœFáWJâîGVZöôÉêKü-
QXρτûΔñëæłíäøãßÎóŒőžõōŵÈÅúıɔεŚÇÄÿåÆòÜűÀḫāᵃḏīšṣḍṭẓᶜġķū'ÊËβ□ɑčÂÔŸÙÏÖδσýṯḫÛřɣṇŋŭŽşºμб'ṭăẽőğŁź!ļŧ𝕏|Á

Cela permet au passage d'obtenir les occurences des différentes lettres des mots du corpus :

```py
```

[2]https://fr.wiktionary.org/wiki/Wiktionnaire:Page_d%E2%80%99accueil
[3]https://dumps.wikimedia.org/
[4]https://kaikki.org/frwiktionary/Fran%C3%A7ais/index.html

```
{'a': 1995983, 'c': 712237, 'u': 796649, 'e': 2434376, 'i': 2156071, 'l': 932081, 'r': 1910391, 'n': 1604191, 'y': 114789, 'o': 1425578, 'p': 467700, 'é': 800569, 'd': 537064, 'm': 507640, 'g': 344961, 'v': 186214, 's': 2272695, 'è': 74466, 'h': 314790, 'f': 242278, 't': 1398597, 'b': 318231, 'A': 18869, 'j': 34770, 'k': 43590, 'C': 29126, 'D': 9935, '-': 166847, 'R': 11733, 'O': 6116, 'M': 25333, 'q': 97362, 'w': 16973, 'E': 7812, 'x': 53222, 'T': 11714, 'I': 3472, 'N': 7797, 'B': 25665, 'ç': 13149, 'à': 2225, 'L': 19822, 'ï': 12065, 'z': 204844, 'H': 8924, 'P': 17469, 'U': 1840, 'S': 29442, 'Y': 990, 'ù': 42, 'œ': 2901, 'F': 9949, 'á': 378, 'W': 4364, 'J': 4295, 'â': 103237, 'î': 10797, 'G': 12696, 'V': 12628, 'Z': 1320, 'ö': 967, 'ô': 6123, 'É': 2342, 'ê': 10617, 'K': 4036, 'ü': 1229, 'Q': 1288, 'X': 348, 'ρ': 1, 'τ': 2, 'û': 4495, 'Δ': 1, 'ñ': 433, 'ë': 1395, 'æ': 1103, 'ɬ': 24, 'í': 300, 'ä': 367, 'ø': 15, 'ã': 53, 'ß': 314, 'Î': 94, 'ó': 376, 'Œ': 26, 'ô᾽': 1, 'ž': 9, 'õ': 7, 'ō': 211, 'ŵ': 5, 'È': 17, 'Å': 1, 'ú': 91, 'ɪ': 18, 'ɔ': 16, 'ɛ': 20, 'Ś': 1, 'Ç': 11, 'Ä': 2, 'ÿ': 33, 'å': 12, 'Æ': 23, 'ò': 49, 'Ü': 18, 'ő': 2, 'À': 4, 'ḫ': 1, 'ā': 55, '᾽': 15, 'ḏ': 1, 'ī': 30, 'š': 29, 'ṣ': 7, 'ḍ': 3, 'ṭ': 2, 'ẓ': 1, '᾽᾽': 2, 'ġ': 1, 'ḳ': 2, 'ū': 33, '᾽᾽᾽': 11, 'Ê': 3, 'Ë': 1, 'β': 14, '□': 1, 'α': 7, 'č': 12, 'Â': 6, 'Ô': 3, 'Ÿ': 1, 'Ù': 1, 'Ï': 2, 'Ö': 24, 'δ': 1, 'σ': 6, 'ý': 6, 'ṯ': 1, 'ḥ': 2, 'Û': 1, 'ř': 5, 'ɣ': 14, 'ṇ': 3, 'ŋ': 10, 'ŭ': 3, 'Ž': 4, 'ş': 3, 'º': 9, 'μ': 2, 'ɓ': 2, '᾽᾽᾽᾽': 10, 'ṭ': 5, 'ă': 10, 'êˇ': 1, 'ő': 3, 'ğ': 9, 'Ł': 7, 'ź': 1, '!': 2, 'ḷ': 1, 'ι': 3, 'Ⲭ': 1, 'I': 9, 'Á': 9, 'Í': 5, 'Ó': 3, 'ŝ': 5, 'ɨ': 3, 'Ò': 17, 'ũ': 1, 'ω': 3, 'ń': 8, 'Š': 5, 'ṃ': 2, 'ộ': 1, 'ľ': 1, 'ʊ': 4, 'ś': 4, 'β': 4, 'ì': 24, 'ʔ': 8, 'ǫ': 1, 'Ú': 4, 'ș': 11, 'ň': 1, 'ď': 2, 'ŷ': 5, 'ė': 2, 'ű': 1, 'ć': 15, 'ť': 1, 'a': 3, 'ę': 8, 'ə': 20, 'ˈˈ': 1, 'ɲ': 2, 'ð': 7, 'Þ': 1, 'ʃ': 1, 'ə': 3, 'Ğ': 2, 'Ş': 1, 'e': 4, 'Ż': 2, 'ǰ': 1, 'θ': 2, 'ʷ': 9, 'ϰ': 4, 'ą': 3, 'ʒ': 1, 'χ': 1, 'ʕ': 2, 'ɬ': 1, 'ż': 1, 'Ø': 1, 'ķ': 1, 'Ć': 3, 'Ŧ': 2, 'c': 1, 'λ': 1, 'Ř': 1, 'ᵉ': 1, 'Č': 1, 'ħ': 1, 'ᵭ': 1, 'ʊ': 1, 'þ': 6, 'ɗ': 1, 'Õ': 10, 'ŏ': 2, '⁹': 1, '□': 2, '⅋': 2, 'ỳ': 1}
```

On remarque qu'un certain nombre d'entre eux n'est utilisé qu'une seule fois parmi tous les mots, donc on peut donc prendre la liberté de les enlever pour simplifier ensuite les algorithmes.

Ainsi, le caractère č est présent 12 fois, dans le mot *Čech*, ses dérivés comme *čeština* et ses déclinaisons comme *češtiny*, *češtine*, *češtinu*, *češtinou*. De la même manière, on remarque qu'on nombre conséquent de caractères relèvent de spécificité d'une langue autre que le français, et leur utilisation étant extrêmement spécifique nous pouvons les enlever.

D'ailleurs, la plupart des logiciels de correction orthographiques ne reconnaissent pas ces mots.

3 Approche algorithmique

Dans un premier temps, on souhaite vouloir accéder à n'importe quel mot rapidement

Pour la distance entre deux mots, on utilise la distance de Damerau-Levenshtein, qui prend en compte les transpositions.

6 voyelles avec 5 accents possibles (rien, ´, `, ^, ¨) -> 30 possibilités () 20 consonnes, sans accent -> 20 possibilités tout en majuscule / minuscule -> 2x 100 possibilités quelques caractères additionnels : -, å, ç, ae, oe, ñ, ø, ß, ō

L'utilisation d'une trie directement occupe BEAUCOUP de mémoire, entre 870MB et 920MB (soit environ 110 Mo), et prend entre 4s et 8s à charger.

La nouvelle implémentation en binaire avec mmap, se charge instantanément et occupe 15 KB de mémoire environ (source: pympler).

Concernant les performances, il lit, corrige et affiche L'Avare de Molière en 1.5s (26828 mots) et corrige le tout en 1.25s.

C'est moins rapide que le système précédent (0.005s vs 0.0002s pour 100 mots et 0.15s vs 2.2s pour L'Avare) soit à peu près 20x plus lent, mais tout à fait acceptable.

On obtient un résultat assez intéressant même si il est perfectible :

```
>>> Bongeour le monde !
Bongeour le monde !
>>> Dodu est-il daudu ?
Dodu est-il daudu ?
>>> Dodu est-île dodu ?
Dodu est-île dodu ?
```

Fig. 1: Correction

En revanche il fonctionne assez bien sur des textes beaucoup plus longs, tel *L'Avare* de Molière.

Ensuite, cela permet de calculer quelques statistiques sur le corpus.

- Il contient à présent 1859114 mots
- Il y a 2129931 noeuds avec un seul enfant dans la trie sur 4034131 noeuds
- Les mots font en moyenne 11.4 caractères
- Chaque noeud a en moyenne 2.2 enfants
- Il y a 1243295 feuilles
- La profondeur moyenne des feuilles est 11.98
- Le mot le plus fait 85 caractères, il s'agit du mot maori *Taumatawhakatangihangakoauauotamateaturipukakapikimaungahoronukupokaiwhenuakitanatahu*, qui signifie *« Le sommet où Tamatea, l'homme aux gros genoux, qui dévalait, avalait et grimpait des montagnes, le marcheur invétéré, joua de sa flûte à un être cher »* désignant une colline en Nouvelle-Zélande
- Les occurences de chaque caractère pour les noeuds
- Les occurences de chaque caractère dans les mots finaux
- Les occurences des bigrammes dans les mots
- Les occurences des trigrammes dans les mots
- Les préfixes avec plus de 10 ou 4 enfants

Puis on peut implémenter assez rapidement un algorithme de correction avec la distance de Damerau-Levenshtein, avec de la programmation dynamique sur l'arbre. On note l'apparition éventuelle de suggestions duppliquées lorsqu'il y a plusieurs manières d'arriver au même mot en une édition, mais on s'en débarasse aisément.



Fig. 2: Correction et suggestions

L'amélioration de la nouvelle structure, avec la fusion des noeuds simples (pour former des noeuds longs) permet de réduire l'espace nécessaire en passant de 32 Mo à 23 Mo, ainsi que les performances (0.7s pour L'Avare), grâce à mémoization de la racine également, temps qui passe à 0.4s si l'on mémoize le premier niveau de la trie (0.3s si on se passe d'afficher le texte dans le terminal). Cela fait 26828 mots en 0.3s, soit à peu près 90000 mots/seconde, ce qui est tout à fait respectable.

Enlever certaines données inutiles (comme l'offset des feuilles) permet de passer ensuite de 23 Mo à 18 Mo (presque x0.5 !), et il n'y a plus que 2222668 noeuds (et heureusement le nombre de feuilles n'a pas changé…)

On peut ensuite potentiellement faire usage des n-grammes pour repérer la phonologie potentielle d'un mot écrit incorrectement et repérer les mots proches.

Une autre approche pour tester l'appartenance d'un mot sont est le Bloom filter[5] (qui peux donner de faux positifs, mais jamais de faux négatifs) ou le BK Tree[6].

Réaliser de manière purement algorithmique un programme qui détecte la correction grammaticale d'une phrase est très difficile.

Par exemple, les phrases (en anglais) ci-dessous sont toutes correctes, avec une position du mot *only* changeant légèrement le sens.

Only Luci told Claude that she loved him.
Luci only told Claude that she loved him.
Luci told only Claude that she loved him.
Luci told Claude only that she loved him.
Luci told Claude that only she loved him.
Luci told Claude that she only loved him.
Luci told Claude that she loved only him.
Luci told Claude that she loved him only.

[5]https://en.wikipedia.org/wiki/Bloom_filter
[6]https://en.wikipedia.org/wiki/BK-tree

4 Utilisation d'un réseau de neurones

Il existe différents types de réseaux de neurones, en fonction du type de tâche à réaliser. Les critères nécessaire pour réaliser un correcteur grammatical sont:

- entrées de taille variable
-

Ceux nous intéressant sont les suivants :

- Réseau de neurones récurrents (RNNs) et en particulier les réseaux récurrent à mémoire courte de long terme (LSTMs) ou les unités récurrentes fermées (GRU)
- Transformeur, plus récent, plus rapide à entraîner et actuellement plus « populaire » que les RNN car ne sont pas obligé de commencer par le début du texte.

4.1 Tokenisation & Embedding

La première étape consiste à encoder l'ensemble des mots du corpus avant de le donner au réseau neuronal. Il faut que cette tokenisation soit cohérente afin de permettre de

- conserver la catégorie grammaticale du mot pour repèrer les erreurs grammaticales (agencement des mots)
- prendre en compte le sens pour affiner la suggestion de mots au vue de la phrase
- conserver l'orthographe/la phonologie du mot pour repèrer des mots proche phonologiquement et orthographiquement

5 Conclusion

6 Annexe

Quelques références intéressantes pour éclaircir ou approfondir certains points.

- La faute de l'orthographe [3]

Bibliographie

[1] V. L. S. d. Lima, « Contribution à l'étude du traitement des erreurs au niveau lexico-syntaxique dans un texte écrit en français », 1990. Consulté le: 3 avril 2025. [En ligne]. Disponible sur: http://www.theses.fr/1990GRE10034

[2] « Centre National de Ressources Textuelles et Lexicales ». Consulté le: 2 avril 2025. [En ligne]. Disponible sur: https://www.cnrtl.fr/

[3] Arnaud Hoedt et Jérôme Piron, *La faute de l'orthographe*, (2019). Consulté le: 2 avril 2025. [En ligne Vidéo]. Disponible sur: https://www.youtube.com/watch?v=5YO7Vg1ByA8